

Java & Spring Boot Interview Question Bank

Real Questions from Real Interviews

A curated collection of **180+ interview questions** covering:

Java Core, OOP, Concurrency, Spring Boot, Microservices,
System Design, Algorithms, and Databases.

Designed for developers with **3 - 5 years of experience**
preparing for backend and full-stack roles.

April 2026

Java Core & Fundamentals

1. What are the main features of Java, and how do they make it different from other programming languages?
2. How do you compile and run a Java program from the command line, and what exactly happens during the compilation and execution phases?
3. Why is Java considered platform-independent, and what role do the JVM and bytecode play in achieving this?
4. What is the role of `public` , `static` , and `void` keywords in `public static void main(String[] args) ?`
5. What is the difference between `==` and `equals()` in Java?
6. How does exception handling work in Java, and what is the difference between checked exceptions, unchecked exceptions, and errors?
7. What happens if we declare a `StringBuffer` as `final` , and can its value still change?
8. Which Java versions have you worked on, and what key features did you use from those versions?
9. How would you reverse a string without using a temporary variable or inbuilt methods?
10. What is the difference between an array and an `ArrayList` in terms of memory usage, performance, and use cases?
11. Have you used `StringBuffer` , and in which scenarios is it appropriate?
12. Why should we avoid using String concatenation (`+`) instead of `StringBuilder.append()` in loops?

Java Core & Fundamentals (Continued)

13. Given a sentence, how do you extract all words into an array?
14. How do you handle multiple exceptions in a single `try-catch` block in Java when there are exceptions like `IOException` , `FileNotFoundException` , and a generic `Exception` ?
15. If a user can input any type of value such as a string, character, or object, which data structure would you use in Java (array or `ArrayList`), and why?
16. You are given an array containing strings and special characters. How do you filter out only the string values in Java?
17. What is a `ClassLoader` in Java, what are its types (Bootstrap, Extension, and Application), and how does the class-loading process work internally?
18. How is memory managed in Java, and what are the responsibilities of stack memory, heap memory, metaspace, and the program counter?
19. What is garbage collection, how do heap generations work, what are the commonly used GC algorithms, and why are `Xms` and `Xmx` important?
20. How does the JVM decide when to perform garbage collection?
21. What is immutability in Java, and why is it important in real-world applications?
22. How do you create a truly immutable class in Java, and what rules must be followed?

Java Core & Fundamentals (Advanced)

23. What is serialization in Java, and what are the best practices and common pitfalls associated with it?
24. If class A throws a runtime exception and class B extends class A and also declares throwing an exception, will the code compile, and why?
25. Even if compilation is successful, why might a Java program throw a `ClassNotFoundException` at runtime?
26. What is a marker interface, and what are some real-world examples of its usage?
27. Why should we not prefer `StringBuffer` in most modern Java applications?
28. Will using `String` objects for millions of records create memory or performance issues, and why?
29. Java 8 features: list the main ones and explain the benefits with a code example.
30. Why is `equals()` + `hashCode()` critical in real systems?
31. If a top-level class is declared as `private` and it contains a `public static void main(String[] args)` method, can this main method be executed by the compiler or JVM?
32. Have you ever created a custom annotation in Java? How do you create a custom annotation?

Java Core & Fundamentals (Advanced Continued)

33. Is it possible to declare a constructor as `final` in Java? Why or why not?
34. What is a `final` class in Java, what restrictions does it impose, and in what real-world scenarios would you intentionally design a class as `final` ?
35. What is covariant method overriding in Java?
36. Are you familiar with Java 21 features, and which ones have you used?
37. Explain the internal working of `HashMap` .
38. How does `HashMap` handle collisions? (`LinkedList` vs `Tree` structure in Java 8+)
39. Why is the default load factor of `HashMap` set to 0.75?
40. What is the Java Memory Model?
41. Explain thread safety and concurrency concepts in Java.
42. How do JVM arguments impact application performance, and what are some commonly used JVM tuning parameters?
43. How do you identify a memory leak in a running Java application?
44. When would you choose `CompletableFuture` over `ExecutorService` ?
45. When should you avoid using `synchronized` and prefer `Lock` ?
46. Why are `String` s immutable in Java? Explain this design decision from both security and performance perspectives.
47. What does a Java Stream actually do internally, and how does it differ conceptually and technically from traditional loops?

Object-Oriented Programming (OOP)

48. What are the four OOP principles, and how can you explain them using real-world examples?
49. What are abstract classes, and in which scenarios should they be used?
50. What is the difference between an abstract class and an interface, and when would you choose one over the other?
51. How do you compare custom objects in Java, and what is the difference between `Comparable` and `Comparator` ?
52. Can an interface implement a class, and why is that not allowed in Java?
53. What is the diamond problem in Java, and how does Java solve it?
54. Why must equal objects always have the same hash code?
55. Why is it important to override `hashCode()` when overriding `equals()` ?
56. For a `Student(id, name)` class, can two students with the same name be considered equal, and how would you design `equals()` and `hashCode()` properly?
57. What are default methods in interfaces, and why were they introduced in Java?
58. If a bank has multiple branches using the same interface but only the Mumbai branch needs special behavior, how would you design this using interfaces or default methods?
59. If new functionality must be added for all branches, how would you implement it without breaking existing code?
60. Is method overriding resolved at compile time or runtime in Java? Explain how dynamic method dispatch works internally.
61. What are `equals()` and `hashCode()` methods in a Java class, how are they used internally by collections, and why are they critical in real-world systems?
62. Why were default methods introduced in interfaces in Java 8, and what problem were they intended to solve?

Multithreading, Concurrency & Async Processing

63. What is the difference between `HashMap` and `ConcurrentHashMap` ?
64. Which data structure is used internally by `HashMap` ?
65. What is a `ThreadPool` , and why should we prefer it over manually creating threads?
66. What is the difference between extending `Thread` and implementing `Runnable` ?
67. How does Java manage threads and memory, and which data is shared versus thread-local?
68. What is a `Lock` in Java, and how is it different from the `synchronized` keyword?
69. What is the `volatile` keyword, and what guarantees does it provide?
70. Can `volatile` alone ensure thread safety, and why or why not?
71. What is `ConcurrentHashMap` , and how does it achieve thread safety internally?
72. How does the internal working of `HashMap` and `ConcurrentHashMap` differ?
73. How would you create a deadlock using two threads?
74. How can you enable and use asynchronous methods in a Spring Boot application?
75. What are `CompletableFuture` s in Java, and how do they help in writing asynchronous and non-blocking code?
76. When multiple threads share resources, how does `wait()` differ from `run()` in terms of behavior and purpose?
77. How do you safely unlock a lock to avoid deadlocks or thread starvation?
78. If two maps are accessed concurrently, should we use separate synchronized blocks, and why?
79. How would you monitor and manage async tasks in Spring Boot, including tracking progress and handling failures?
80. How have you implemented multithreading in your projects, and how did it improve performance, scalability, or responsiveness?

Design Patterns

81. What are the commonly used design patterns in Java, and where are they applied?
82. What is the Singleton design pattern, what are its different implementations, and which one is preferred in production?
83. Why is the constructor private in a Singleton, and how is the instance created?
84. What is the Decorator pattern, and when should it be used?
85. Which microservices design patterns have you worked with?
86. In the Factory pattern, how do you decide which subclass should create the object?
87. In what ways can the Singleton design pattern be broken in Java (for example, using reflection or serialization), and how can these issues be fixed?
88. What are the SOLID principles, and how would you explain them using real-world banking examples?
89. What is the Circuit Breaker pattern, and why is it important in microservices architecture?
90. What is the Half-Open state in the Circuit Breaker pattern, and what does it represent?
91. What is the Saga pattern, and how does Choreography differ from Orchestration?

Spring Framework & Spring Boot (Core)

92. How does Spring Boot simplify deployment compared to traditional Spring applications?
93. What is Inversion of Control (IoC), and how does Dependency Injection work in Spring?
94. How are beans created and managed in the Spring container?
95. What is the default bean scope in Spring?
96. Which annotations are used to create beans in Spring?
97. What is the difference between `@Primary` and `@Qualifier` ?
98. What is `@Profile` , and when should it be used?
99. What is the lifecycle of a bean?
100. How do you handle exception handling in REST controllers in Spring Boot?
101. What is the internal lifecycle of a Spring bean?
102. How do you handle circular dependencies in Spring?
103. How can you prevent auto-configuration when a custom bean already exists?
104. What are correlation IDs, and why are they used?
105. How do you fetch the correlation ID in a Java Spring Boot application?
106. How do you set an existing correlation ID in Spring Boot?
107. How do you create a new correlation ID in a Spring Boot application?
108. Can a Java Spring Boot project connect to two different databases at the same time?
109. How do you connect to multiple databases in a Spring Boot application?
110. What is the difference between `@Component`, `@Service` , and `@Repository` in Spring?
111. Explain bean scopes and the Spring bean lifecycle.
112. Describe the Spring Boot application startup sequence.
113. How does `@Transactional` work internally, and what are common pitfalls?
114. Design a rate limiter that allows a maximum of 5 requests per user per minute.
115. When a file is uploaded from a frontend application, how do you read the binary data in a Spring Boot controller, pass it to the service layer, and persist it in the database?
116. What is Spring WebFlux, and how does it work? How do you use it in a Spring Boot application?
117. How would you configure an Amazon S3 client in a Spring Boot application, including credential management and environment-specific configuration?

118. How would you configure a Redis client in a Spring Boot application, and how would you ensure that the cache is updated only after a database transaction is successfully committed?
119. How would you configure and manage two different databases in a single Spring Boot application, including datasource routing and transaction management?

Spring Boot – Web, REST & Error Handling

120. How do you call one service from another in Spring Boot?
121. What are the commonly used Controller layer annotations?
122. What are the commonly used Service layer annotations?
123. How would you design a simple Spring Boot API to fetch student details, and what layers would you create?
124. What is `ResponseEntity` , and why is it used?
125. Is it good practice to return a JSON body with `ResponseEntity` , and why?
126. What are the alternatives to `ResponseEntity` ?
127. How do you handle 404 errors in Spring Boot?
128. How do you implement pagination in Spring Boot?
129. What is the difference between PUT, POST, and PATCH?
130. How does Spring Boot support internationalization (i18n)?
131. What are the core REST API design principles you follow?
132. What happens if the same PUT or POST request is sent multiple times?
133. How would you implement date validation when a request is received from a client as a request body in a Spring Boot REST API?
134. You are working with an incremental database where today there are 1000 records, tomorrow there will be 1050 records, and the day after that there will be 1085 records, with previous records being repeated and only a few new ones added each day. In this scenario, which HTTP method would you prefer PUT, PATCH, or POST and why?
135. How would you ensure that custom error messages are returned to the client while preventing internal or sensitive error details from being exposed?
136. How would you implement pagination in a Spring Boot application, and what considerations would you keep in mind for performance and scalability?

Spring Boot – Security

- 137. How would you implement user login and authentication in a Spring Boot application?
- 138. What authentication mechanisms would you consider while designing login functionality?
- 139. What is the difference between OAuth and JWT ?
- 140. How does token expiration work in authentication systems?
- 141. Can JWT tokens be invalidated? If yes, how?
- 142. Explain the refresh token flow.
- 143. How would you configure Spring Security to secure endpoints using form-based or token-based authentication?

CI/CD, DevOps & Logging

144. What are GitHub Actions, and how are they used in a CI/CD workflow?
145. Have you ever created a CI/CD pipeline using Jenkins, and can you explain how you would design and configure a Jenkins pipeline for a Spring Boot application?
146. What is the correct way to implement logging in a Java Spring Boot project, which logging framework would you use, and how would you log a failure in a specific method so that it can be effectively traced and debugged?
147. How do you ensure that application logs are written in a proper and consistent sequence, especially in a multi-threaded or distributed environment?
148. How would you log the total execution time of a method in a Spring Boot application, and what tools or techniques would you use?

Microservices & Distributed Systems

149. How do microservices communicate with each other?
150. What is the ideal response time for banking microservices?
151. What are the disadvantages of microservices, and when might they not be the right choice?
152. How does an API Gateway route incoming client requests to the appropriate backend microservices, and what factors are involved in this routing process?
153. What is the ideal time for a Java service to access the repository for read versus write operations?
154. How would you implement API retry logic in a Spring Boot application so that a failed method or external API call is retried one or two times before failing permanently?
155. How do you ensure that two APIs can communicate with each other, and in a Spring Boot application, how would you call a third-party REST API reliably?
156. What challenges did you face while migrating from a monolith to microservices?
157. How would you optimize an API taking 4 seconds to respond?
158. What is the Circuit Breaker pattern, and why is it critical in distributed systems?
159. What are the major pain points when migrating from a monolithic Spring Boot application to a microservices architecture, and what challenges arise when moving in the opposite direction?

Algorithms & Coding Problems

160. Given a string `"banana"`, how would you count the number of occurrences of each character using Java Streams?
161. How would you find the first non-repeating character in a string like `"Mississippi"` ?
162. How would you use Streams to find the second maximum number?
163. How would you find duplicate numbers while maintaining insertion order?
164. Given a `String[]` input, how would you filter the array using only Java Streams such that only strings starting with `"A"` are selected and converted to uppercase?
165. Given a sentence, how do you extract all words into an array?
166. Given a string containing `{`, `}`, and `*`, where `*` can represent `{`, `}`, or an empty string, determine whether the expression is balanced.
167. Given a list of words and an integer `k`, return the top `k` most frequent words ordered by frequency. If frequencies are equal, sort them lexicographically.
168. Reverse a linked list in pairs. Input: `1 → 2 → 3 → 4 → 5` Output: `2 → 1 → 4 → 3 → 5` .
169. When are Streams faster than loops, and when should loops be preferred?
170. What is the difference between DFS and BFS, and when would you use each in real-world systems like banking?
171. How would you design an LRU Cache, and what data structures would you use?
172. How would you design and implement a custom `LinkedList` in Java such that elements are always inserted in sorted (ascending) order?

SQL & Databases

- 173. Write a query to count employees by city and gender.
- 174. What are inner joins and outer joins in SQL?
- 175. Write a query to find the employee with the second-highest salary in Bangalore.
- 176. How would you optimize a slow SQL query?
- 177. What role do indexed columns play in `WHERE` and `JOIN` clauses?
- 178. Why should `SELECT *` be avoided in production queries?
- 179. How do you use `EXPLAIN` to analyze query performance?
- 180. When would you consider denormalization?
- 181. How do you analyze slow query logs?
- 182. What are the benefits of NoSQL databases over SQL databases, and in what scenarios would you prefer one over the other?